

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO  
CIÊNCIA DA COMPUTAÇÃO

LEANDRO BROGNOLI GRAZZIOTIN E VICTOR DA ROCHA TONIATO

**ANALISADOR LÉXICO**  
TEORIA DA COMPUTAÇÃO E LINGUAGENS FORMAIS

SÃO MATEUS - ES  
2026

## Sumário

|                                                |    |
|------------------------------------------------|----|
| <b>1 INTRODUÇÃO</b>                            | 3  |
| <b>2 LINGUAGEM UTILIZADA</b>                   | 3  |
| 2.1 EXPRESSÕES REGULARES PARA OS <i>TOKENS</i> | 3  |
| 2.2 MÁQUINA DE MOORE                           | 4  |
| <b>3 TESTES DO ANALISADOR LÉXICO</b>           | 5  |
| <b>4 DIAGRAMA DE CLASSES</b>                   | 7  |
| <b>5 CÓDIGO IMPLEMENTADO</b>                   | 7  |
| 5.1 AnalisadorLexico.h                         | 8  |
| 5.2 AnalisadorLexico.cpp                       | 9  |
| 5.3 ErroLexico.h                               | 10 |
| 5.4 ErroLexico.cpp                             | 11 |
| 5.5 MaquinaMoore.h                             | 12 |
| 5.6 MaquinaMoore.cpp                           | 12 |
| 5.7 Constantes.h                               | 17 |
| 5.8 Main.cpp                                   | 18 |
| <b>6 CONCLUSÃO</b>                             | 20 |

## 1 INTRODUÇÃO

A Máquina de Moore é um autômato finito determinístico (AFD) com saídas associadas aos estados. O presente trabalho busca reconstruir a etapa de análise léxica de um compilador, para isso, ela recebe uma sequência de caracteres de entrada e classifica cada lexema reconhecido em um *token*.

## 2 LINGUAGEM UTILIZADA

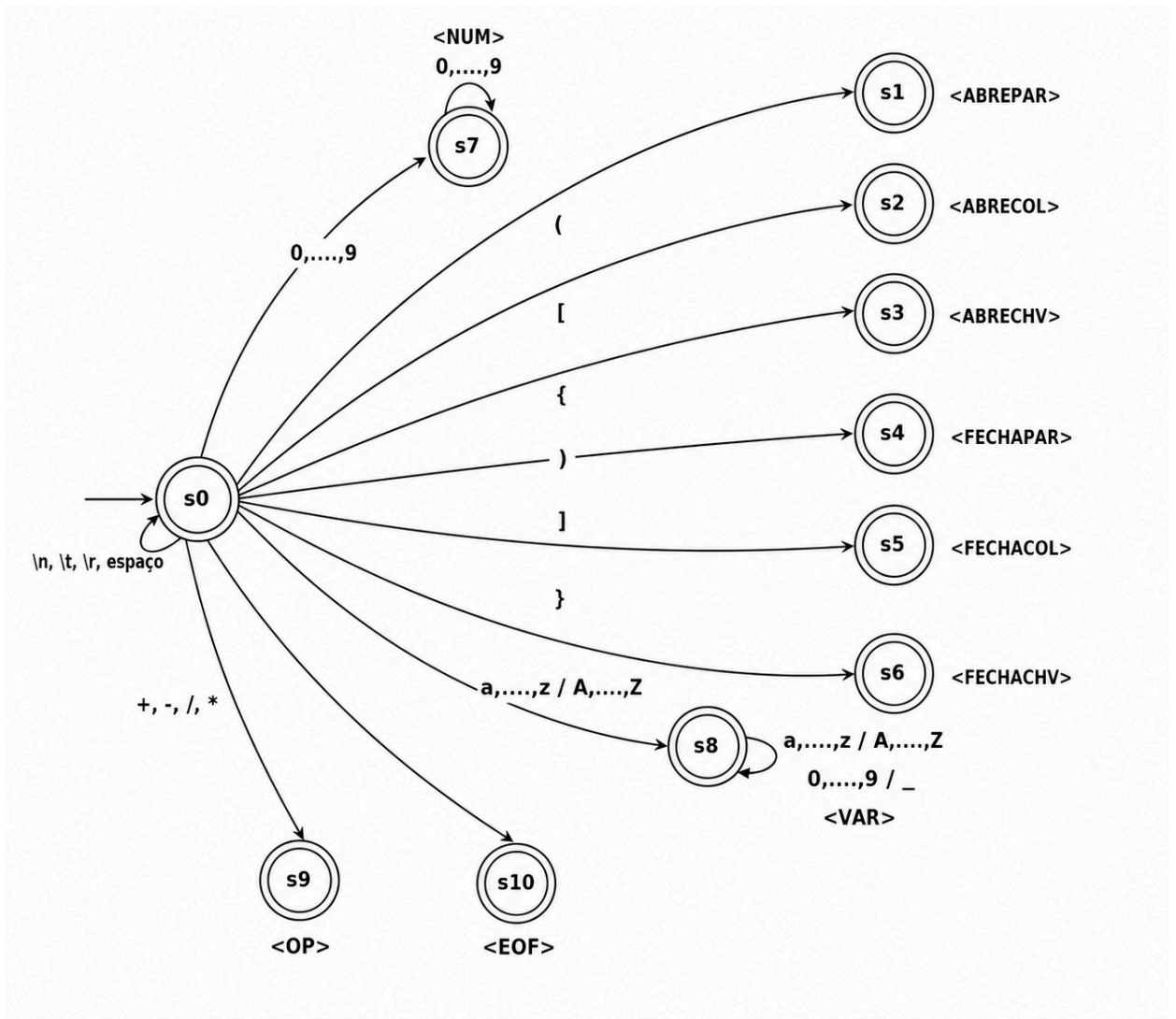
Para esse trabalho, suponha uma linguagem L contendo expressões com parênteses, colchetes e chaves (todos balanceados), as quatro operações básicas, números e variáveis. Sendo os números inteiros positivos e as variáveis contendo letras, números e `'_'` (*underscore*). A exceção é que as variáveis não podem iniciar por número ou por `'_'`.

### 2.1 EXPRESSÕES REGULARES PARA OS TOKENS

A seguir estão dispostas, em tabela, os *tokens* associados a cada expressão regular e exemplos de lexemas.

| TOKENS     | ER                                                                              | EXEMPLOS        |
|------------|---------------------------------------------------------------------------------|-----------------|
| <ABREPAR>  | ( '(' )                                                                         | (, ((, (((      |
| <ABRECOL>  | ( '[' )                                                                         | [, [[, [[[      |
| <ABRECHV>  | ( '{' )                                                                         | {, {{, {{{      |
| <FECHAPAR> | ( ')' )                                                                         | ), ),, )))      |
| <FECHACOL> | ( ']' )                                                                         | ], ],, ]]]      |
| <FECHACHV> | ( '}' )                                                                         | }, },, }}}      |
| <NUM>      | (1, ... , 9) . (0, ... , 9)*                                                    | 1, 35, 80, 999  |
| <VAR>      | (a   ...   z   A   ...   Z) . ( '_'   a   ...   z   A   ...   Z   0   ...   9)* | Ab, a_12, VrT20 |
| <OP>       | ( +   -   /   * )                                                               | +, -, /, *      |
| <EOF>      | caractere que sinaliza o fim do arquivo                                         |                 |

## 2.2 MÁQUINA DE MOORE



## 3 TESTES DO ANALISADOR LÉXICO

Teste 1:

x + 25

**Console de saída:**

<VAR>

<OP>

<NUM>

<EOF>

Análise léxica realizada com sucesso para o arquivo teste1.txt

Teste 2:

```
( xyz / 45 )  
{ Teste_42 }
```

**Console de saída:**

<ABREPAR>

<VAR>

<OP>

<NUM>

<FECHAPAR>

<ABRECHV>

<VAR>

<FECHACHV>

<EOF>

Análise léxica realizada com sucesso para o arquivo teste2.txt

Teste 3:

```
[ ) _proibido
```

**Console de saída:**

<ABRECOL>

<FECHAPAR>

Erro léxico: caractere encontrado: \_

Era(m) esperado(s):

123456789\_abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ012

3456789+/\*{[()]} )

Teste 4:

```
{ ab_33_ )  
10 / @
```

**Console de saída:**

<ABRECHV>

<VAR>

<FECHAPAR>

<NUM>

<OP>

Erro léxico: caractere encontrado: @

Era(m) esperado(s):

123456789\_abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ012

3456789+/\*{[()]} )

Teste 5:

Brasil 1 x 1 Marrocos

{ [ ( triste ) ] }

Japao vem forte

**Console de saída:**

<VAR>

<NUM>

<VAR>

<NUM>

<VAR>

<ABRECHV>

<ABRECOL>

<ABREPAR>

<VAR>

<FECHAPAR>

<FECHACOL>

<FECHACHV>

<VAR>

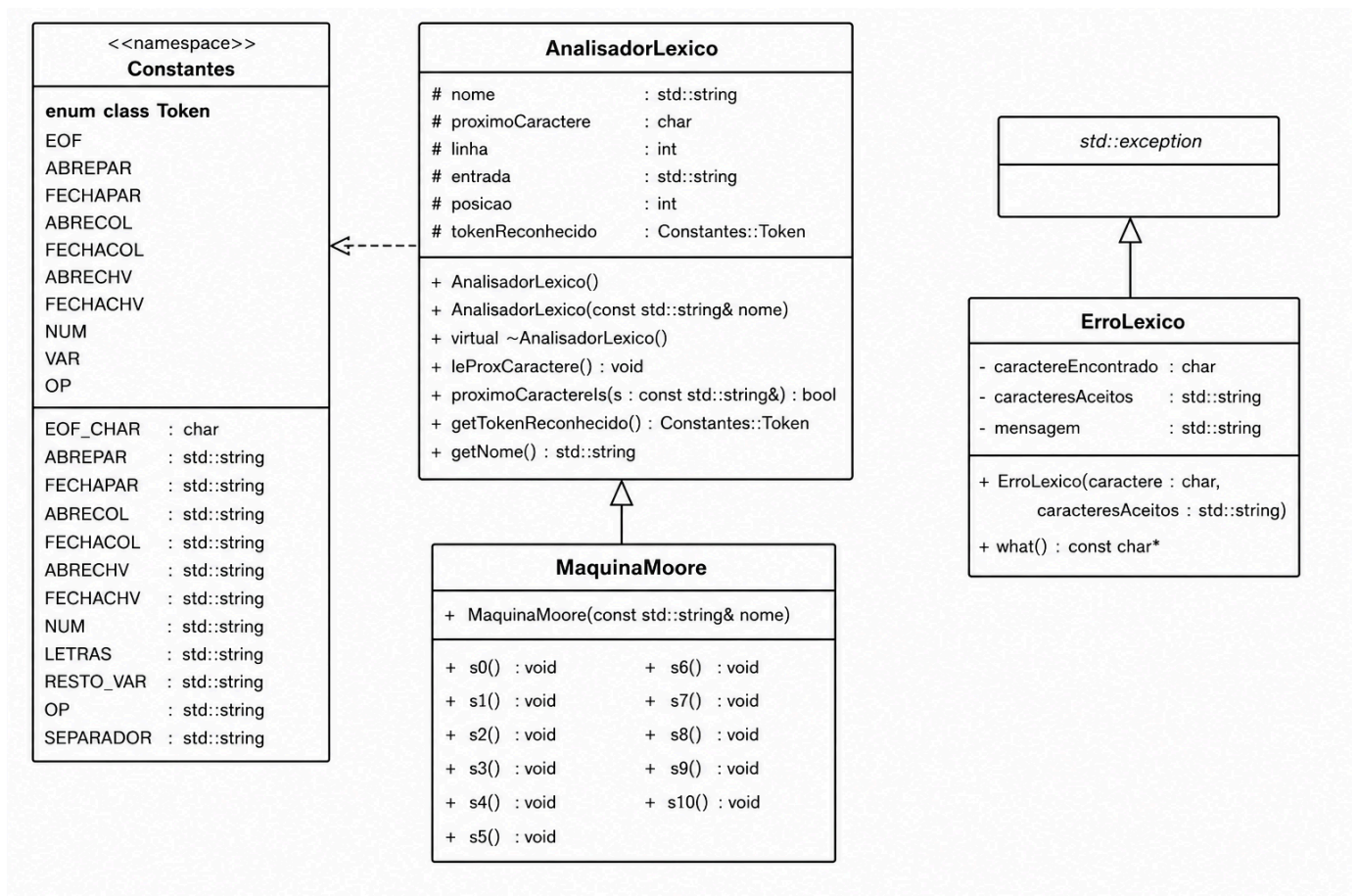
<VAR>

<VAR>

<EOF>

Análise léxica realizada com sucesso para o arquivo teste5.txt

## 4 DIAGRAMA DE CLASSES



## 5 CÓDIGO IMPLEMENTADO

Vale ressaltar que o trabalho foi desenvolvido inicialmente em Java, porém a implementação a seguir foi feita em linguagem C++. Para acessar o conteúdo separado por pastas, classes e os arquivos de teste, acesse o link a seguir para o projeto no [GitHub](#).

Ademais, foi desenvolvido o arquivo Makefile com o objetivo de facilitar a compilação e execução do programa. Para isso, deve-se utilizar os comandos abaixo, recomenda-se utilizar somente o 'make valgrind-full' que irá compilar, executar, limpar e verificar erros.

- make -> compila o código
- make run -> executa o código
- make valgrind -> executa e verifica erros no código

- make clean -> limpa os arquivos objeto e executáveis
- make full -> compila, executa e limpa arquivos objeto e executáveis
- make valgrind-full -> compila, executa, limpa e verifica erros

Todos esses comandos devem ter o arquivo analisado ao lado (ex: make valgrind-full teste1.txt).

## 5.1 AnalisadorLexico.h

```

1  #ifndef ANALISADORLEXICO_H
2
3  #define ANALISADORLEXICO_H
4
5  #include <string>
6  #include "Constantes.h"
7
8  class AnalisadorLexico {
9  protected:
10     std::string nome;
11     char proximoCaractere;
12     int linha;
13     std::string entrada;
14     int posicao;
15     Constantes::Token tokenReconhecido;
16
17 public:
18     explicit AnalisadorLexico(const std::string& nome);
19     AnalisadorLexico();
20
21     void leProxCaractere();
22
23     bool proximoCaractereIs(const std::string& s) const;
24
25     Constantes::Token getTokenReconhecido() const;
26
27     std::string getNome() const;
28
29     virtual ~AnalisadorLexico() = default;
30 };
31
32 #endif

```



## 5.2 AnalisadorLexico.cpp

```
1  #include "AnalisadorLexico.h"
2
3  #include <fstream>
4  #include <stdexcept>
5
6  AnalisadorLexico::AnalisadorLexico(const std::string& nome)
7      : nome(nome),
8        proximoCaractere(Constants::EOF_CHAR),
9        linha(1),
10        entrada(""),
11        posicao(0)
12  {
13      std::ifstream arquivo(nome);
14
15      if (!arquivo.is_open()) {
16          throw std::runtime_error(
17              "Erro de leitura no arquivo " + nome
18          );
19      }
20
21      char c;
22
23      while (arquivo.get(c)) {
24          entrada += c;
25      }
26
27      arquivo.close();
28
29      leProxCaractere();
30  }
31
32  AnalisadorLexico::AnalisadorLexico()
33      : AnalisadorLexico("entrada.txt")
34  {
35  }
36
37  void AnalisadorLexico::leProxCaractere() {
38      if (posicao < static_cast<int>(entrada.size())) {
39
40          proximoCaractere = entrada[posicao];
41
42          if (proximoCaractere == '\n') {
43              linha++;
44          }
45      }
```

```

46         posicao++;
47     }
48     else {
49         proximoCaractere = Constantes::EOF_CHAR;
50     }
51 }
52
53 ✓ bool AnalisadorLexico::proximoCaractereIs(
54     const std::string& s) const {
55
56     return s.find(proximoCaractere) != std::string::npos;
57 }
58
59 Constantes::Token
60 AnalisadorLexico::getTokenReconhecido() const {
61     return tokenReconhecido;
62 }
63
64 std::string AnalisadorLexico::getNome() const {
65     return nome;
66 }

```

### 5.3 ErroLexico.h

```

1  #ifndef ERROLEXICO_H
2  #define ERROLEXICO_H
3
4  #include <exception>
5  #include <string>
6
7  ✓ class ErroLexico : public std::exception {
8      private:
9          char caractereEncontrado;
10         std::string caracteresAceitos;
11         std::string mensagem;
12
13     public:
14         ErroLexico(char caractereEncontrado,
15             const std::string& caracteresAceitos);
16
17         const char* what() const noexcept override;
18     };
19
20 #endif

```

## 5.4 ErroLexico.cpp

```
1  #include "ErroLexico.h"
2
3  ✓ ErroLexico::ErroLexico(char caractereEncontrado,
4                          const std::string& caracteresAceitos)
5      : caractereEncontrado(caractereEncontrado),
6        caracteresAceitos(caracteresAceitos)
7  {
8      std::string caractere;
9
10     switch (this->caractereEncontrado) {
11
12         case '\n':
13             caractere = "\\n";
14             break;
15
16         case '\r':
17             caractere = "\\r";
18             break;
19
20         case '\t':
21             caractere = "\\t";
22             break;
23
24         case '\0':
25             caractere = "EOF";
26             break;
27
28         default:
29             caractere = std::string(1, this->caractereEncontrado);
30     }
31
32     mensagem =
33         "Caractere não pertencente à linguagem: "
34         + caractere
35         + "\nOs caracteres aceitos são: "
36         + this->caracteresAceitos;
37 }
38
39 const char* ErroLexico::what() const noexcept {
40     return mensagem.c_str();
41 }
```

## 5.5 MaquinaMoore.h

```
1  #ifndef MAQUINAMOORE_H
2  #define MAQUINAMOORE_H
3
4  #include "AnalizadorLexico.h"
5
6  class MaquinaMoore : public AnalizadorLexico {
7  public:
8      explicit MaquinaMoore(const std::string& nome);
9
10     void s0();
11     void s1();
12     void s2();
13     void s3();
14     void s4();
15     void s5();
16     void s6();
17     void s7();
18     void s8();
19     void s9();
20     void s10();
21 };
22
23 #endif
```

## 5.6 MaquinaMoore.cpp

```
1  #include "MaquinaMoore.h"
2  #include "ErroLexico.h"
3
4  MaquinaMoore::MaquinaMoore(const std::string& nome)
5      : AnalizadorLexico(nome)
6  {
7  }
8
9  void MaquinaMoore::s0() {
10
11     while (proximoCaractereIs(Constants::SEPARADOR)) {
12         leProxCaractere();
13     }
14
15     if (proximoCaractereIs(Constants::ABREPAR)) {
16         leProxCaractere();
17         s1();
18     }
```

```

20     else if (proximoCaractereIs(Constants::ABRECOL)) {
21         leProxCaractere();
22         s2();
23     }
24
25     else if (proximoCaractereIs(Constants::ABRECHV)) {
26         leProxCaractere();
27         s3();
28     }
29
30     else if (proximoCaractereIs(Constants::FECHAPAR)) {
31         leProxCaractere();
32         s4();
33     }
34
35     else if (proximoCaractereIs(Constants::FECHACOL)) {
36         leProxCaractere();
37         s5();
38     }
39
40     else if (proximoCaractereIs(Constants::FECHACHV)) {
41         leProxCaractere();
42         s6();
43     }
44
45     else if (proximoCaractereIs(Constants::NUM)) {
46         leProxCaractere();
47         s7();
48     }
49
50     else if (proximoCaractereIs(Constants::LETRAS)) {
51         leProxCaractere();
52         s8();
53     }
54
55     else if (proximoCaractere == Constants::EOF_CHAR) {
56         leProxCaractere();
57         s10();
58     }

```

```

59
60     else if (proximoCaractereIs(Constants::OP)) {
61         leProxCaractere();
62         s9();
63     }
64
65     else {
66         throw ErroLexico(
67             proximoCaractere,
68             Constants::NUM
69             + Constants::RESTO_VAR
70             + Constants::OP
71             + Constants::ABRECHV
72             + Constants::ABRECOL
73             + Constants::ABREPAR
74             + Constants::FECHACHV
75             + Constants::FECHACOL
76             + Constants::FECHAPAR
77         );
78     }
79 }
80
81 void MaquinaMoore::s1() {
82     tokenReconhecido = Constants::Token::ABREPAR;
83 }
84
85 void MaquinaMoore::s2() {
86     tokenReconhecido = Constants::Token::ABRECOL;
87 }
88
89 void MaquinaMoore::s3() {
90     tokenReconhecido = Constants::Token::ABRECHV;
91 }
92
93 void MaquinaMoore::s4() {
94     tokenReconhecido = Constants::Token::FECHAPAR;
95 }

```

```

97     void MaquinaMoore::s5() {
98         tokenReconhecido = Constantes::Token::FECHACOL;
99     }
100
101     void MaquinaMoore::s6() {
102         tokenReconhecido = Constantes::Token::FECHACHV;
103     }
104
105     void MaquinaMoore::s7() {
106
107         tokenReconhecido = Constantes::Token::NUM;
108
109         if (proximoCaractereIs(Constantes::NUM)) {
110
111             leProxCaractere();
112             s7();
113         }
114
115         else if (!(proximoCaractereIs(Constantes::SEPARADOR)
116             || proximoCaractere == Constantes::EOF_CHAR
117             || proximoCaractereIs(Constantes::ABREPAR)
118             || proximoCaractereIs(Constantes::ABRECHV)
119             || proximoCaractereIs(Constantes::ABRECOL)
120             || proximoCaractereIs(Constantes::FECHAPAR)
121             || proximoCaractereIs(Constantes::FECHACOL)
122             || proximoCaractereIs(Constantes::FECHACHV)
123             || proximoCaractereIs(Constantes::OP))) {
124
125             throw ErroLexico(
126                 proximoCaractere,
127                 "separador ou fim de Token"
128             );
129         }
130     }
131
132     void MaquinaMoore::s8() {
133
134         tokenReconhecido = Constantes::Token::VAR;

```

```

135
136     if (proximoCaractereIs(Constants::RESTO_VAR)) {
137
138         leProxCaractere();
139         s8();
140     }
141
142     else if (!(proximoCaractereIs(Constants::SEPARADOR)
143         || proximoCaractere == Constants::EOF_CHAR
144         || proximoCaractereIs(Constants::ABREPAR)
145         || proximoCaractereIs(Constants::ABRECHV)
146         || proximoCaractereIs(Constants::ABRECOL)
147         || proximoCaractereIs(Constants::FECHAPAR)
148         || proximoCaractereIs(Constants::FECHACOL)
149         || proximoCaractereIs(Constants::FECHACHV)
150         || proximoCaractereIs(Constants::OP))) {
151
152         throw ErroLexico(
153             proximoCaractere,
154             "separador ou fim de Token"
155         );
156     }
157 }
158
159 void MaquinaMoore::s9() {
160     tokenReconhecido = Constants::Token::OP;
161 }
162
163 void MaquinaMoore::s10() {
164     tokenReconhecido = Constants::Token::EOF_TOKEN;
165 }

```



## 5.7 Constantes.h

```
1  #ifndef CONSTANTES_H
2
3  #define CONSTANTES_H
4
5  #include <string>
6
7  namespace Constantes {
8
9  ✓  enum class Token {
10      EOF,
11      ABREPAR,
12      FECHAPAR,
13      ABRECOL,
14      FECHACOL,
15      ABRECHV,
16      FECHACHV,
17      NUM,
18      VAR,
19      OP
20  };
21
22  const char EOF_CHAR = '\0';
23
24  const std::string ABREPAR = "(";
25  const std::string FECHAPAR = ")";
26  const std::string ABRECOL = "[";
27  const std::string FECHACOL = "]";
28  const std::string ABRECHV = "{";
29  const std::string FECHACHV = "}";
30
31  const std::string NUM = "0123456789";
32
33  const std::string LETRAS =
34      "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
35
36  const std::string RESTO_VAR =
37      "_abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
38
39  const std::string OP = "+-/*";
40
41  const std::string SEPARADOR = "\t\r\n ";
42  }
43
44  #endif
```

## 5.8 Main.cpp

```
1  #include <iostream>
2  #include <stdexcept>
3
4  #include "MaquinaMoore.h"
5  #include "ErroLexico.h"
6
7  ✓ int main(int argc, char* argv[]) {
8
9      if (argc < 2) {
10         std::cout << "Uso: " << argv[0]
11             << " <arquivo_entrada>" << std::endl;
12         return 1;
13     }
14
15     try {
16         MaquinaMoore scanner(argv[1]);
17
18         do {
19             scanner.s0();
20
21             switch (scanner.getTokenReconhecido()) {
22
23                 case Constantes::Token::ABREPAR:
24                     std::cout << "ABREPAR";
25                     break;
26
27                 case Constantes::Token::FECHAPAR:
28                     std::cout << "FECHAPAR";
29                     break;
30
31                 case Constantes::Token::ABRECOL:
32                     std::cout << "ABRECOL";
33                     break;
34
35                 case Constantes::Token::FECHACOL:
36                     std::cout << "FECHACOL";
37                     break;
38
39                 case Constantes::Token::ABRECHV:
40                     std::cout << "ABRECHV";
41                     break;
```

```

42
43         case Constantes::Token::FECHACHV:
44             std::cout << "FECHACHV";
45             break;
46
47         case Constantes::Token::NUM:
48             std::cout << "NUM";
49             break;
50
51         case Constantes::Token::VAR:
52             std::cout << "VAR";
53             break;
54
55         case Constantes::Token::OP:
56             std::cout << "OP";
57             break;
58
59         case Constantes::Token::EOF_TOKEN:
60             std::cout << "EOF";
61             break;
62     }
63
64     std::cout << std::endl;
65
66     } while (scanner.getTokenReconhecido()
67             != Constantes::Token::EOF_TOKEN);
68
69     std::cout << "Análise léxica do arquivo "
70               << scanner.getNome()
71               << " realizada com sucesso!"
72               << std::endl;
73 }
74
75 catch (const ErroLexico& erro) {
76     std::cout << "===== ERRO LÉXICO ====="
77               << std::endl
78               << erro.what()
79               << std::endl;
80 }
81
82 catch (const std::exception& erro) {
83     std::cout << "===== ERRO ====="
84               << std::endl
85               << erro.what()
86               << std::endl;
87 }
88
89 return 0;
90 }

```

## **6 CONCLUSÃO**

Durante o desenvolvimento, foram aplicados conceitos de autômatos finitos, análise léxica e modelagem UML orientada a objetos, o que contribuiu para uma melhor organização do código e compreensão do funcionamento interno do analisador. Os testes realizados demonstraram que a solução é capaz de reconhecer os tokens definidos pela linguagem e tratar situações de erro de forma adequada, atendendo aos requisitos propostos e consolidando os conhecimentos estudados sobre a etapa de análise léxica na construção de compiladores.